# PROJECT DOCUMENTATION

## Node.js and React-Based Web Application

| | |
|---|---|
| **Project Title:** | PULSE — Enterprise Glassmorphic Analytics Platform |
| **Submitted By:** | Yash Pawar |
| **Roll Number:** | 291 |
| **Institute Name:** | Guru Nanak Khalsa College |
| **Academic Year:** | 2026 – 2027 |

Technology Stack: React.js • Node.js • Express.js • MongoDB • Vite

## 1. Introduction

Modern business intelligence and operational management require intuitive, visually responsive, and secure digital dashboards. Traditional dashboards often present rigid, uninspiring user interfaces with high latency in data presentation and disjointed workflows between metrics visualization, report synthesis, and data exportation.

**PULSE** is a full-stack web application architected to address these shortcomings. It combines an ultra-modern **glassmorphism aesthetic** (supporting dual Dark and Light mode themes) with a high-throughput **Node.js and Express.js RESTful backend** and **MongoDB with Mongoose ODM**. The platform delivers real-time operational KPI tracking, interactive time-series vector charts, searchable report data grids, instant CSV/JSON report exports, and role-based access control (RBAC).

## 2. Objectives

- **Develop a User-Friendly Web Application**: Deliver an exceptional user experience using modern glassmorphic UI principles (translucent backdrop blur, ambient neon glows, and responsive typography).
- **Build a Responsive Frontend Using React.js**: Implement component modularity with React 19, Vite for lightning-fast bundling and Hot Module Replacement (HMR), and custom CSS design systems without relying on rigid utility frameworks.
- **Create Scalable Backend Services and APIs**: Build a clean RESTful service in Node.js and Express.js supporting authentication, data aggregation, and report lifecycle management.
- **Process User Requests Efficiently and Securely**: Implement stateless JSON Web Token (JWT) authorization, bcrypt password hashing, input validation, and asynchronous database queries with MongoDB.
- **Enable Data Portability**: Provide automated data transformations allowing users to filter, inspect, and export analytical reports into structured CSV and JSON files on demand.

## 3. Technology Stack

- **Frontend**: React.js (v19), Vite (v8), Vanilla CSS (CSS Custom Variables, Glassmorphism Backdrop Blur, Flexbox/Grid), Lucide React Icons, Google Fonts (*Outfit* & *Plus Jakarta Sans*).
- **Backend**: Node.js (v24 LTS), Express.js (v4.21), JWT ( `jsonwebtoken` ), `bcryptjs` , `cors` , `dotenv` .
- **Database**: MongoDB (Port 27017 / MongoDB Atlas Cloud), Mongoose ODM (v8.9) with automated schema validation and lifecycle hooks.

- **Development Tools**: Visual Studio Code, Antigravity IDE, Git & GitHub ( `yashwebsite` repository).

## 4. System Requirements

### Hardware Requirements

- **Processor**: Intel Core i3 / AMD Ryzen 3 or higher (Minimum 2.0 GHz).
- **Memory (RAM)**: Minimum 4 GB RAM (8 GB recommended for concurrent dev servers).
- **Storage**: Minimum 500 MB free hard drive space.
- **Display**: 1366 × 768 screen resolution or higher.

### Software Requirements

- **Operating System**: Windows 10/11, macOS, or Linux.
- **Node.js & npm**: Node.js v18.0.0+ (v24 installed) and npm v10+.
- **Database**: MongoDB Community Server v6.0+.
- **Web Browser**: Chromium or WebKit browser (Chrome, Edge, Firefox, Brave).

## 5. System Architecture

The application adopts a decoupled, multi-tier client-server architecture:

> **Architecture Workflow:**
> [Client: React 19 + Vite] ↔ (RESTful HTTP/JSON with JWT) ↔ [API Gateway: Express.js Port 5000]
> ↓
> [Middleware: Auth & Validation] → [Controllers: Auth, Dashboard, Reports] → [Mongoose ODM] → [MongoDB Port 27017]

- **Client Tier**: The React frontend executes in the browser, managing UI state, theme selection (Dark/Light), JWT token storage in `localStorage`, and reactive DOM rendering.
- **REST API Tier**: Express.js routes incoming HTTP requests to dedicated controllers (`authController`, `dashboardController`, `reportController`).
- **Middleware Layer**: Validates request payloads, verifies Bearer tokens via `authMiddleware`, and provides centralized error handling.
- **Data Persistence Tier**: Mongoose ODM handles mapping between JavaScript objects and MongoDB collections, executing asynchronous aggregation pipelines and queries.

## 6. Project Structure

```
pulse-app/
├── package.json                # Monorepo root script orchestration
├── README.md                   # Project documentation & execution guide
├── implementation_plan.md      # Architecture blueprint & schema spec
├── .gitignore                  # Ignores node_modules, build dist, and secrets
├── backend/
│   ├── config/db.js            # MongoDB connection handler via Mongoose
│   ├── controllers/            # authController, dashboardController, reportController
│   ├── middleware/             # authMiddleware (JWT verification & RBAC)
│   ├── models/                 # User.js (bcrypt hook), Report.js, Activity.js
│   ├── routes/                 # authRoutes, dashboardRoutes, reportRoutes
│   ├── seed/seedData.js        # Database mock seeder script
│   ├── .env.example            # Environment template
│   ├── package.json            # Backend dependencies & start scripts
│   └── server.js               # Express entry point (Port 5000)
└── frontend/
    ├── public/favicon.svg      # Cyan vector pulse icon
    ├── src/
    │   ├── components/
    │   │   ├── common/         # GlassCard, Navbar, Sidebar, ThemeToggle, Modal, Toast
    │   │   ├── auth/           # LoginForm, RegisterForm
    │   │   ├── dashboard/      # StatCard, MetricChart, ActivityFeed, QuickActions
    │   │   └── reports/        # ReportTable, ReportFilters, CreateReportModal
    │   ├── context/            # AuthContext.jsx & ThemeContext.jsx
    │   ├── services/api.js     # REST API client with Bearer token injection
    │   ├── App.css             # Layout & animation rules
    │   ├── App.jsx             # Main view router & application shell
    │   ├── index.css           # Glassmorphism design tokens & typography
    │   └── main.jsx            # React DOM entry point
    ├── index.html              # HTML5 shell with Google Fonts
    ├── package.json            # Frontend dependencies (React, Vite, Lucide)
    └── vite.config.js          # Vite development server configuration
```

## 7. Module Description

- **Authentication Module**: Handles user registration, credential validation, and stateless session management with role-based access control (*Admin*, *Manager*, *Analyst*). Returns signed JWTs and records audit logs.
- **Executive Dashboard Module**: Aggregates real-time organizational KPIs (Total Reports, Active Team Members, Success Rate, Total Exports). Delivers interactive SVG time-series throughput and latency charts (7D, 30D, 90D).
- **Reports & Data Repository Module**: Manages analytical documents, real-time keyword search, category pills (*Financial*, *Operational*, *Marketing*, *System*), expandable row details with custom KPI tags, and instant CSV/JSON exports.
- **Report Synthesis Module**: Modal dialog enabling authorized operators to publish reports with custom key-value metrics (e.g. `arr: $4.8M`, `throughput: 1,800 req/s`) directly into

MongoDB.

- **Security & Audit Feed Module**: Maintains an immutable, chronological event stream of all system actions (logins, report generation, data exports, role modifications).

## 8. Implementation Details

### Backend Implementation

- **Database Connection (`config/db.js`)**: Asynchronously connects to MongoDB using `mongoose.connect()` with connection state monitoring.
- **Password Security (`models/User.js`)**: Pre-save hook hashes passwords using `bcryptjs` with salt rounds of 10. Exposes `matchPassword()` for validation.
- **JWT Authentication (`middleware/authMiddleware.js`)**: Validates `Bearer <token>` in the `Authorization` header using `jwt.verify()` and injects authenticated user data into `req.user`.
- **Data Aggregation (`controllers/dashboardController.js`)**: Executes MongoDB aggregation pipelines to calculate totals and status distributions.
- **CSV Export Streaming (`controllers/reportController.js`)**: Formats document records into RFC 4180 CSV strings and sets HTTP headers for automated browser file download.

### Frontend Implementation

- **Glassmorphism Design System (`src/index.css`)**: Uses CSS variables for frosted background surfaces, border glows, and `backdrop-filter: blur(16px) saturate(180%)`.
- **Dual-Theme Engine (`context/ThemeContext.jsx`)**: Toggles root `data-theme` attribute (Dark/Light) with instant CSS variable re-binding and `localStorage` persistence.
- **Interactive Vector Charting (`components/dashboard/MetricChart.jsx`)**: Mathematically generates cubic Bezier curves in native SVG with glowing area gradient fills and coordinate-mapped hover tooltips.
- **1-Click Demo Switcher (`components/common/Navbar.jsx`)**: Pre-configured login shortcuts for immediate evaluation as *Alexander Vance* (Admin) or *Sarah Connor* (Analyst).

## 9. Screenshots & Visual Interface

| Screen View | Description & Visual Features |
|---|---|
| **Executive Dashboard (Dark)** | High-contrast dark mode with glowing ambient cyan/purple orbs, 4 frosted KPI stat cards, dynamic SVG area chart, and operator quick actions. |
| **Executive Dashboard (Light)** | Crisp, glassmorphic light aesthetic with subtle frosted drop-shadows and vibrant accent tags. |
| **Reports Repository** | Filterable table showing category badges, status pills, author metadata, and action buttons for CSV and JSON downloads. |
| **Expanded Report Details** | In-place accordion expansion displaying executive synthesis narrative and extracted metric chips (e.g., ARR, Uptime, P99 Latency). |
| **Report Creation Modal** | Frosted glass dialog with dynamic key-value KPI metric builder, live validation, and category selectors. |
| **Authentication Modal** | Glass login and registration dialog with 1-click quick demo role access buttons. |

## 10. Testing

| Test ID | Feature Tested | Test Steps | Expected Result | Status |
|---|---|---|---|---|
| **TC-01** | API Health Check | Send GET /api/health | HTTP 200 with status: "online" | **PASS** |
| **TC-02** | User Registration | Submit form with valid credentials | User created in DB, JWT token returned | **PASS** |
| **TC-03** | User Authentication | Submit valid credentials for admin@pulse.io | HTTP 200, JWT returned, user profile set in state | **PASS** |
| **TC-04** | Invalid Login | Submit incorrect password | HTTP 401 with error message displayed | **PASS** |
| **TC-05** | Theme Toggling | Click ThemeToggle button in Navbar | Switches data-theme between dark and light | **PASS** |
| **TC-06** | Dashboard Stats | Fetch /api/dashboard/stats | Populates KPI cards with live MongoDB metrics | **PASS** |
| **TC-07** | Timeframe Filtering | Click 7D, 30D, and 90D chart buttons | SVG re-renders curve coordinates for interval | **PASS** |
| **TC-08** | Real-Time Search | Type "Revenue" in reports search bar | Table filters to show only matching reports | **PASS** |
| **TC-09** | Report Synthesis | Fill modal form and submit report | Report stored in MongoDB, activity logged, table refreshed | **PASS** |
| **TC-10** | CSV Data Export | Click CSV icon on report row | Browser downloads report-<id>.csv file | **PASS** |
| **TC-11** | Report Deletion | Authenticate as admin & click trash icon | Report deleted from MongoDB, toast displayed | **PASS** |

## 11. Results

The implemented application satisfies 100% of the project objectives. Vite bundling completes in 3.03 seconds; API endpoints respond in under 25 milliseconds. The UI provides a fluid glassmorphism aesthetic across mobile and desktop, while the backend maintains strict separation of concerns and data integrity.

## 12. Conclusion

The **PULSE** platform showcases the power of combining Node.js, Express.js, MongoDB, and React.js to create modern, responsive, and secure web applications. The project reinforced key skills in full-stack architecture, REST API design, JWT authentication, SVG data visualization, and glassmorphic UI design.

## 13. Future Scope

- Automated scheduled reporting via cron workers and email delivery.
- Native server-side PDF generation using Puppeteer.
- WebSocket real-time telemetry for multi-operator collaborative updates.

## 14. References


1. React.js Documentation: https://react.dev/
2. Node.js Documentation: https://nodejs.org/docs/
3. Express.js API Reference: https://expressjs.com/
4. MongoDB & Mongoose: https://mongoosejs.com/
5. GitHub Repository: https://github.com/yashpawar0610-lab/yashwebsite.git